Tecnológica del Oriente
Programa de Ingeniería de Software

Electiva II (Inteligencia Artificial Avanzada) 20262 - N6

Estudiante
Ruben Dario Bayona Galvis

Actividad 4. Creación de un modelo de inteligencia artificial avanzada

Tutor: Leydi Johana Polo Amador

Medellin

10/11/2026

Contenido

1. Introducción y justificación del problema 4
2. Revisión de literatura 5
3. Diseño del modelo y metodología 7
4. Resultados de entrenamiento y evaluación 9
5. Discusión sobre equidad y ética 12
6. Conclusiones y recomendaciones 14
Referencias 15

Resumen

Automuelles Virtual es una plataforma de venta de repuestos de suspensión (muelles, amortiguadores, espirales, bujes, terminales y rótulas). Hoy el cliente que no sabe qué falla tiene su vehículo depende de visitar el taller o de que un asesor interprete su descripción por WhatsApp, lo que produce ventas equivocadas, devoluciones y tiempos de atención altos. Este trabajo diseña, entrena y evalúa un clasificador multiclase que, a partir de los datos del vehículo, los síntomas marcados y una descripción libre, propone las tres fallas más probables y los repuestos asociados. Se compararon regresión logística, Random Forest y LightGBM; LightGBM se ajustó con optimización bayesiana (Optuna, 60 pruebas). Sobre un conjunto de prueba independiente de 750 casos, el modelo final obtuvo una exactitud de 67,7 %, un F1-macro de 0,665 y una exactitud top-3 de 92,5 %. El análisis de equidad con Fairlearn muestra un desempeño menor en vehículos de carga y de servicio público que en particulares; la re-ponderación por grupo no cerró la brecha, lo que apunta a la calidad y cantidad de los datos de esos grupos como causa principal. Los datos son semisintéticos, por lo que los resultados validan la metodología y no el desempeño en producción. Se propone desplegar el sistema como apoyo a la decisión, con confirmación obligatoria de un técnico.

1. Introducción y justificación del problema

1.1 Contexto

Automuelles Virtual es el canal digital de un negocio colombiano de repuestos y servicios de suspensión. Atiende a conductores de vehículos particulares, camionetas, taxis y busetas de servicio público y vehículos de carga liviana y mediana. La mayoría de sus clientes no son expertos en mecánica: llegan con una molestia ("el carro rebota", "suena un golpe en los huecos", "se ve caído de atrás") y necesitan saber qué pieza comprar.

En el proceso actual el diagnóstico se hace de dos formas: (1) el cliente lleva el vehículo al taller, lo que exige desplazamiento y tiempo, o (2) un asesor interpreta por WhatsApp o teléfono lo que el cliente describe. La segunda vía depende de la experiencia del asesor, no escala con el volumen de consultas y es propensa a errores, porque síntomas parecidos pueden venir de fallas distintas. Un golpe seco en los huecos, por ejemplo, puede deberse a bujes desgastados, a un amortiguador vencido o a una rótula con juego.

1.2 Problema

¿Cómo orientar al cliente de Automuelles Virtual hacia la falla más probable de su suspensión, y hacia el repuesto correcto, antes de la compra y sin la presencia inmediata de un técnico?

El problema tiene consecuencias concretas para el negocio y para el cliente:

- **Ventas equivocadas y devoluciones:** el cliente compra la pieza que cree necesaria; si no era esa, se generan devoluciones, costos logísticos e insatisfacción.
- **Carga operativa:** cada consulta ambigua consume tiempo de un asesor experto, que es un recurso escaso.
- **Seguridad vial:** una suspensión en mal estado aumenta la distancia de frenado y reduce la estabilidad. Orientar a tiempo al conductor tiene un valor que va más allá de la venta.

1.3 Por qué una solución de inteligencia artificial

El diagnóstico a partir de síntomas es un problema de **clasificación supervisada multiclase**: dadas unas variables de entrada (vehículo, uso, síntomas y texto libre) se debe asignar una de seis categorías de falla. Las reglas fijas del tipo "si rebota, es el amortiguador" no capturan las interacciones entre variables (un vehículo de carga con sobrepeso que se ve bajo apunta a muelles; un particular con el mismo síntoma apunta a espirales), mientras que un modelo de aprendizaje automático las aprende de los casos históricos. Además, un modelo probabilístico entrega una **lista ordenada de hipótesis con su probabilidad**, lo que permite mostrar al cliente las tres fallas más probables y decidir cuándo el caso es demasiado incierto y debe pasar a un técnico.

1.4 Objetivos

Objetivo general: desarrollar y evaluar un modelo de aprendizaje automático que entregue un diagnóstico preliminar de fallas de suspensión y la recomendación de repuestos asociada, integrable en la plataforma Automuelles Virtual.

1. Construir un conjunto de datos representativo del problema y documentar sus supuestos y limitaciones.
2. Comparar modelos de distinta complejidad y ajustar los hiperparámetros del modelo principal sobre un conjunto de validación.
3. Evaluar el modelo final con métricas de clasificación y de utilidad para el negocio (exactitud top-3) en un conjunto de prueba independiente.
4. Analizar el desempeño por subgrupos de vehículos para detectar sesgos, y probar una técnica de mitigación.
5. Exponer el modelo mediante una API REST con un formulario de demostración.

2. Revisión de literatura

2.1 Aprendizaje automático para diagnóstico automotriz

Theissler et al. (2021) revisan el uso de aprendizaje automático para mantenimiento predictivo en la industria automotriz y distinguen entre enfoques basados en señales de sensores (vibración, telemetría) y enfoques basados en datos de servicio y reportes de talleres. Señalan como retos recurrentes la escasez de datos etiquetados, el desbalance entre clases de falla y la necesidad de que el modelo sea interpretable para el técnico. En el caso de Automuelles Virtual no hay sensores: la información disponible es la que reporta el cliente, por lo que el problema se ubica en el segundo grupo y hereda esos mismos retos.

2.2 Modelos para datos tabulares

Los datos del problema son tabulares (variables categóricas, numéricas y binarias) con un componente de texto corto. Grinsztajn, Oyallon y Varoquaux (2022) y Shwartz-Ziv y Armon (2022) muestran que, en datos tabulares de tamaño medio, los métodos de ensamble de árboles con *gradient boosting* siguen superando o igualando a las redes neuronales profundas, con menor costo de entrenamiento y ajuste. Entre ellos, XGBoost (Chen y Guestrin, 2016) popularizó el *boosting* regularizado, y LightGBM (Ke et al., 2017) introdujo el crecimiento de árboles por hoja y el muestreo basado en gradientes, lo que lo hace más rápido con un desempeño equivalente. Random Forest (Breiman, 2001) es una alternativa robusta basada en *bagging*, y la regresión logística multinomial es la línea base clásica, lineal e interpretable.

Implicación para el diseño: se descartan las redes profundas por el tamaño del conjunto de datos y se comparan tres familias (lineal, *bagging* y *boosting*), con LightGBM como candidato principal.

2.3 Representación de texto libre

La descripción que escribe el cliente se puede representar con TF-IDF (Salton y Buckley, 1988), que pondera cada término por su frecuencia en el documento y su rareza en la colección. Es una técnica simple, rápida y adecuada para textos cortos y vocabularios reducidos; los modelos de lenguaje preentrenados son más potentes, pero su costo no se justifica sin un volumen grande de texto real.

2.4 Ajuste de hiperparámetros

Bergstra y Bengio (2012) demostraron que la búsqueda aleatoria es más eficiente que la búsqueda en malla cuando solo algunos hiperparámetros importan. Optuna (Akiba et al., 2019) va un paso más allá con un muestreador bayesiano (TPE) que concentra las pruebas en las regiones prometedoras del espacio de búsqueda. Por eso se eligió Optuna en lugar de GridSearchCV.

2.5 Desbalance de clases

Las fallas no ocurren con la misma frecuencia. Las técnicas habituales son el sobremuestreo sintético (SMOTE; Chawla et al., 2002) y la ponderación de clases en la función de pérdida. Se optó por la ponderación (`class_weight = balanced`), porque SMOTE interpola entre casos y no es apropiado cuando hay variables categóricas y texto.

2.6 Equidad y sesgo en aprendizaje automático

Mehrabi et al. (2021) clasifican las fuentes de sesgo según la etapa en que aparecen. Las más relevantes aquí son el **sesgo de representación** (algunos grupos tienen pocos ejemplos) y el **sesgo de medición** (las etiquetas de algunos grupos son menos confiables). Barocas, Hardt y Narayanan (2023) proponen evaluar la equidad comparando métricas de error entre grupos, más que buscar una sola métrica global. Fairlearn (Bird et al., 2020) implementa esta evaluación con `MetricFrame`. Como mitigación en la etapa de preprocesamiento, Kamiran y Calders (2012) proponen re-ponderar las muestras para que cada grupo tenga el mismo peso total. Mitchell et al. (2019) recomiendan documentar el modelo en una "ficha de modelo" (*model card*) que declare su uso previsto, sus límites y sus resultados por subgrupo.

2.7 Sistemas de recomendación e interacción humano–IA

Los sistemas de recomendación clásicos (Koren, Bell y Volinsky, 2009) se apoyan en el historial de compras de muchos usuarios. En este caso la recomendación del repuesto se deriva del diagnóstico, es decir, es una recomendación basada en conocimiento: cada falla se asocia con las piezas del catálogo que la resuelven. Amershi et al. (2019) proponen pautas para la interacción humano–IA, entre ellas dejar claro qué tan bien puede hacer el sistema su trabajo y facilitar la corrección humana; estas pautas guían el diseño de la respuesta de la API (probabilidades visibles, advertencia y derivación a un técnico).

Síntesis: la literatura justifica (1) un ensamble de *boosting* para datos tabulares, (2) TF-IDF para el texto corto, (3) ajuste bayesiano de hiperparámetros, (4) ponderación de clases, y (5) una evaluación de equidad por subgrupos con Fairlearn, con mitigación por re-ponderación y supervisión humana.

3. Diseño del modelo y metodología

3.1 Conjunto de datos

Automuelles Virtual no cuenta todavía con un registro histórico digital y etiquetado de diagnósticos. Por eso se construyó un **conjunto de datos semisintético** de 5.000 casos con el script `generar_datos.py`. Cada caso se genera en cuatro pasos:

1. **Vehículo:** se elige el tipo de vehículo con proporciones similares a las de la clientela de un taller de suspensión (55 % particulares, 22 % camionetas, 13 % servicio público y 10 % carga), luego la marca (18 marcas, el 8 % de origen chino) y luego el año, el kilometraje, la carga habitual y el tipo de vía.
2. **Falla:** se asigna con una probabilidad base por tipo de vehículo (por ejemplo, los vehículos de carga usan muelles y los particulares, espirales), ajustada según el uso: más kilometraje aumenta la falla de amortiguadores y bujes, las vías destapadas aumentan la de bujes y terminales, la sobrecarga aumenta la de muelles y espirales, y los vehículos nuevos tienen más casos sin falla.
3. **Síntomas:** cada falla activa nueve síntomas binarios con probabilidades definidas a partir de criterios técnicos de suspensión. Los perfiles se solapan, como ocurre en la realidad.
4. **Ruido:** se genera una descripción en lenguaje coloquial colombiano (vacía en el 27 % de los casos y con errores de digitación ocasionales) y se cambia la etiqueta al azar en el 6 % de los casos (12 % en vehículos de carga), para simular diagnósticos ambiguos o mal registrados.

Esta decisión se declara como **la principal limitación del trabajo**: los resultados demuestran que la metodología funciona, pero el desempeño real solo se conocerá con órdenes de servicio reales. El generador deja todos los supuestos en constantes explícitas, para que un técnico pueda revisarlos y ajustarlos.

Tabla 1. *Variables de entrada y salida del modelo.*

Grupo	Variables	Tipo
Vehículo	tipo de vehículo, marca, origen de la marca	Categórica
Uso	año, kilometraje	Numérica
Uso	carga habitual (baja, media, alta), tipo de vía (urbana, mixta, destapada)	Categórica
Síntomas	rebote excesivo, golpe en huecos, inclinación lateral, vehículo bajo, desgaste irregular de llantas, vibración del volante, juego en la dirección, ruido al girar, fuga de aceite	Binaria (9)
Texto	descripción libre del cliente	Texto
Salida	amortiguador dañado, muelle fatigado, espiral vencido, bujes desgastados, terminal o rótula, sin falla de suspensión	Clase (6)

Tabla 2. *Distribución de clases y de tipos de vehículo en el conjunto de datos (n = 5.000).*

Falla	Casos	Tipo de vehículo	Casos
Amortiguador dañado	1.345	Particular	2.807
Bujes desgastados	1.050	Camioneta	1.073
Espiral vencido	782	Servicio público	640
Terminal o rótula	745	Carga	480
Sin falla de suspensión	645		
Muelle fatigado	433		

3.2 División de los datos

Los datos se dividieron de forma **estratificada por clase** en 3500 casos de entrenamiento (70 %), 750 de validación (15 %) y 750 de prueba (15 %). El conjunto de validación se usó para comparar modelos y ajustar hiperparámetros; el de prueba se usó **una sola vez**, al final, para no sobrestimar el desempeño.

3.3 Preprocesamiento

Todo el preprocesamiento está dentro de un `Pipeline` de scikit-learn (Pedregosa et al., 2011), de modo que se ajusta solo con los datos de entrenamiento y se aplica igual en validación, prueba y producción, lo que evita fugas de información:

- **Variables categóricas:** codificación *one-hot*, ignorando las categorías desconocidas en producción (por ejemplo, una marca nueva).
- **Variables numéricas:** estandarización (media 0, desviación 1).
- **Síntomas:** se pasan sin transformar (ya son 0/1).
- **Texto libre:** TF-IDF con unigramas y bigramas, eliminación de tildes y de palabras vacías del español, frecuencia mínima de 3 documentos y escalado sublineal.
- **Desbalance:** pesos de clase inversos a su frecuencia (`class_weight = balanced`).

3.4 Arquitectura del sistema

El sistema tiene tres capas. (1) **Entrada:** un formulario web donde el cliente indica el vehículo, marca los síntomas y escribe su descripción. (2) **Modelo:** el *pipeline* de preprocesamiento seguido de un clasificador LightGBM, que devuelve una probabilidad por cada una de las seis fallas. (3) **Salida y decisión:** una API REST (FastAPI) que ordena las probabilidades, devuelve las tres fallas más probables con los repuestos del catálogo asociados a cada una, clasifica la confianza (alta ≥ 0,60; media ≥ 0,40; baja < 0,40) y marca el caso para **revisión técnica obligatoria** cuando la confianza es baja o el vehículo es de carga, el grupo en el que el modelo es menos confiable (sección 5).

3.5 Modelos comparados y justificación

- **Regresión logística multinomial (línea base):** modelo lineal, rápido e interpretable; sirve de referencia mínima.

- **Random Forest (400 árboles):** ensamble por *bagging*, robusto al ruido y con pocos hiperparámetros.
- **LightGBM (modelo principal):** *gradient boosting* por hoja, eficiente con matrices dispersas (como las de TF-IDF y *one-hot*), con manejo nativo de pesos y buen desempeño en datos tabulares según la literatura revisada.

3.6 Ajuste de hiperparámetros

Se usó Optuna con el muestreador TPE durante 60 pruebas, maximizando el F1-macro en el conjunto de validación. El espacio de búsqueda incluyó: número de árboles (100–800), tasa de aprendizaje (0,01–0,2, escala logarítmica), número de hojas (8–64), profundidad máxima (3–10), mínimo de muestras por hoja (5–60), submuestreo de filas (0,6–1,0) y de columnas (0,4–1,0) y regularización L2 (0,001–10).

3.7 Métricas de evaluación

- **Exactitud (accuracy):** proporción de diagnósticos correctos.
- **F1-macro:** promedio del F1 de cada clase con el mismo peso, de modo que no se oculte un mal desempeño en las fallas poco frecuentes. Es la métrica principal.
- **Exactitud top-3:** proporción de casos en que la falla real está entre las tres que el sistema muestra. Es la métrica que más se acerca a la experiencia del cliente.
- **Matriz de confusión** normalizada por clase real, para ver qué fallas se confunden entre sí.
- **Métricas por subgrupo** (Fairlearn `MetricFrame`): exactitud y F1-macro por tipo de vehículo, origen de la marca y antigüedad, y la brecha máxima entre grupos.

3.8 Herramientas y reproducibilidad

El proyecto se implementó en Python con scikit-learn 1.7, LightGBM 4.7, Optuna 5.0, Fairlearn 0.14 y FastAPI, y todas las etapas usan la semilla 42. `src/generar_datos.py` genera los datos; `src/entrenar.py` entrena, ajusta y evalúa los modelos y produce las figuras y `reports/resultados.json`; `src/api.py` expone el modelo en `POST /diagnostico` y sirve el formulario de demostración (`uvicorn src.api:app`).

4. Resultados de entrenamiento y evaluación

4.1 Comparación de modelos

La Tabla 3 y la Figura 1 comparan los modelos sobre el conjunto de validación.

Tabla 3. *Desempeño de los modelos en el conjunto de validación (750 casos).*

Modelo	Exactitud	F1-macro	Top-3
Regresión logística (línea base)	0,649	0,639	0,911
Random Forest	0,603	0,586	0,900
LightGBM (sin ajustar)	0,629	0,615	0,889
LightGBM (ajustado con Optuna)	**0,663**	**0,648**	**0,905**

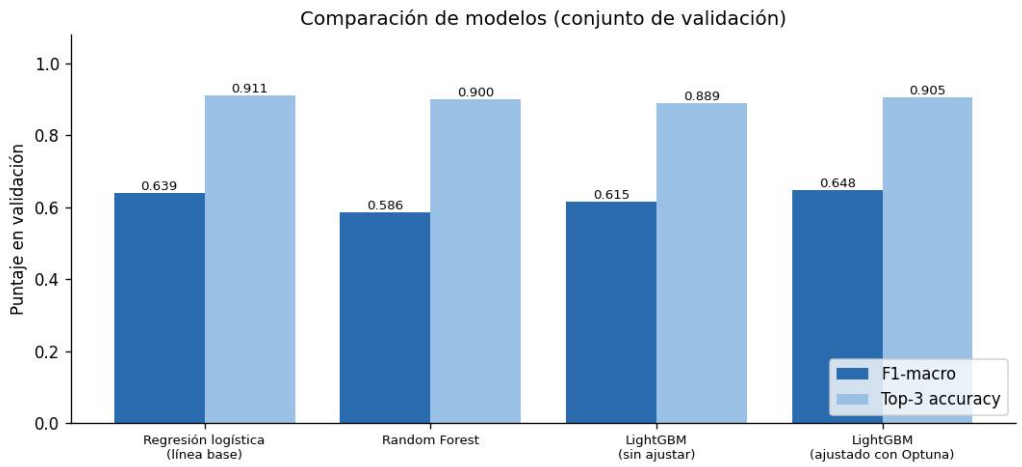


Figura 1. *F1-macro y exactitud top-3 de cada modelo en validación.*

El LightGBM ajustado obtuvo el mejor F1-macro (0,648), pero la ventaja sobre la regresión logística (0,639) es pequeña. Esto es coherente con la forma en que se generaron los datos: los síntomas actúan de forma casi independiente sobre la falla, y esa relación es aproximadamente lineal. Con datos reales, en los que se esperan más interacciones, la ventaja del *boosting* debería crecer. Random Forest quedó por debajo, porque los árboles profundos sin regularizar se ajustan al ruido de las etiquetas.

4.2 Ajuste de hiperparámetros

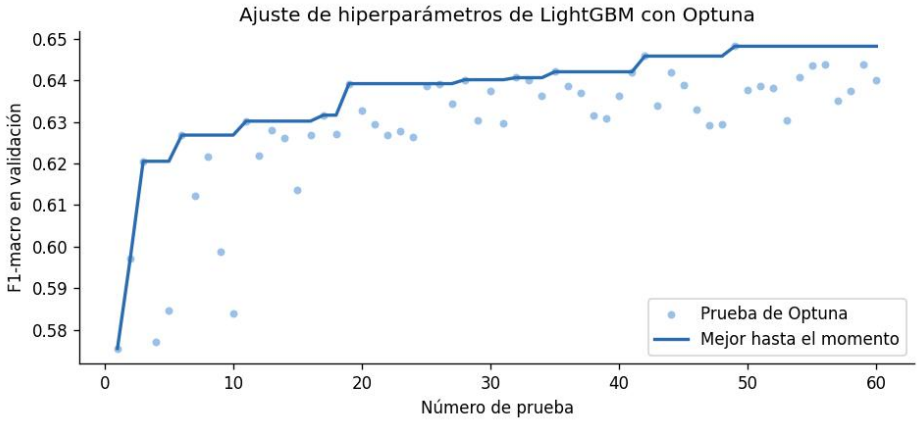


Figura 2. *Evolución del F1-macro en validación durante las pruebas de Optuna.*

El ajuste subió el F1-macro de LightGBM de 0,615 a 0,648 (+0,033). La mejor configuración usa 200 árboles, una tasa de aprendizaje de 0,0101, 10 hojas, una profundidad máxima de 6 y un mínimo de 41 muestras por hoja. Es decir, Optuna eligió un modelo **más simple y más regularizado** que el predeterminado, lo que confirma que el principal riesgo con estos datos era el sobreajuste al ruido.

Estudio de ablación: un LightGBM sin la descripción libre obtuvo un F1-macro de 0,617, prácticamente igual al LightGBM con texto sin ajustar (0,615). En estos datos el texto no añade información, porque se generó a partir de los mismos síntomas marcados. Con descripciones

reales, que suelen mencionar detalles que no están en las casillas, se espera que el texto sí aporte.

4.3 Evaluación final en el conjunto de prueba

Tabla 4. *Métricas del modelo final (LightGBM ajustado) en el conjunto de prueba (750 casos).*

Métrica	Valor
Exactitud	67,7 %
F1-macro	0,665
F1 ponderado	0,674
Exactitud top-3	92,5 %

El desempeño en prueba (F1-macro 0,665) es similar al de validación (0,648), lo que indica que el ajuste no sobreajustó el conjunto de validación. El resultado más relevante para el negocio es la **exactitud top-3 de 92,5 %**: en más de nueve de cada diez consultas, la falla real aparece entre las tres opciones que se le muestran al cliente. Como referencia, un clasificador que adivinara al azar acertaría en el 16,7 % de los casos (y en el 50 % con tres opciones).

Tabla 5. *Precisión, sensibilidad (recall) y F1 por clase en el conjunto de prueba.*

Falla	Precisión	Sensibilidad	F1	Casos
Amortiguador dañado	0,799	0,811	0,805	201
Bujes desgastados	0,625	0,506	0,559	158
Espiral vencido	0,694	0,581	0,633	117
Muelle fatigado	0,649	0,769	0,704	65
Sin falla de suspensión	0,580	0,598	0,589	97
Terminal o rótula	0,622	0,795	0,698	112

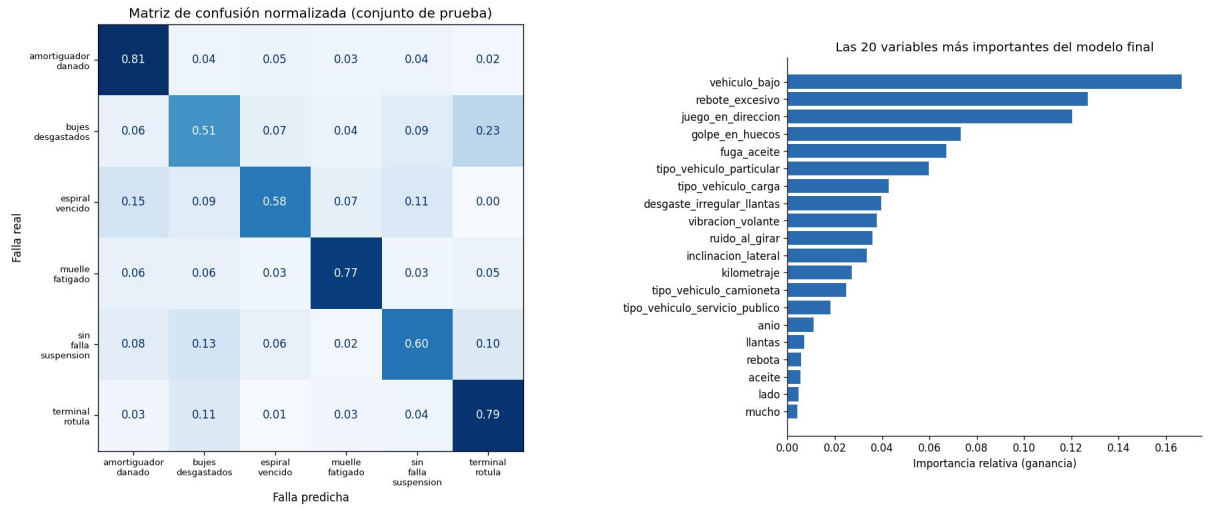


Figura 3. *(a) Matriz de confusión normalizada por clase real (conjunto de prueba). (b) Las 20 variables más importantes del modelo final (ganancia de LightGBM).*

La matriz de confusión (Figura 3a) muestra que la clase mejor reconocida es **amortiguador dañado**, que tiene síntomas distintivos (rebote y fuga de aceite). Las confusiones más frecuentes tienen sentido mecánico: **bujes desgastados** se confunde con **terminal o rótula**, porque ambos producen golpes, ruidos al girar y vibración, y **espiral vencido** se confunde con **amortiguador**, porque ambos producen rebote e inclinación. Estas son, justamente, las confusiones que también comete un asesor sin ver el vehículo, y justifican mostrar tres opciones en lugar de una.

Según la Figura 3b, las variables más importantes son síntomas con un significado técnico claro: *vehículo bajo* (muelles y espirales), *rebote excesivo* (amortiguadores) y *juego en la dirección* (terminales y rótulas), seguidas del tipo de vehículo y el kilometraje. Esto da confianza en que el modelo usa relaciones razonables y no artefactos de los datos.

5. Discusión sobre equidad y ética

5.1 Atributos analizados

En este dominio no se tratan atributos protegidos de personas como el género o la etnia, pero sí hay grupos de clientes que podrían recibir un servicio de peor calidad. Se analizaron tres: el **tipo de vehículo**, porque los conductores de carga y de servicio público suelen ser trabajadores independientes cuyo sustento depende del vehículo; el **origen de la marca** (china frente a tradicional), porque las marcas chinas son recientes en el mercado y tienen poca representación; y la **antigüedad del vehículo**, como aproximación a la capacidad económica del propietario.

5.2 Resultados por subgrupo

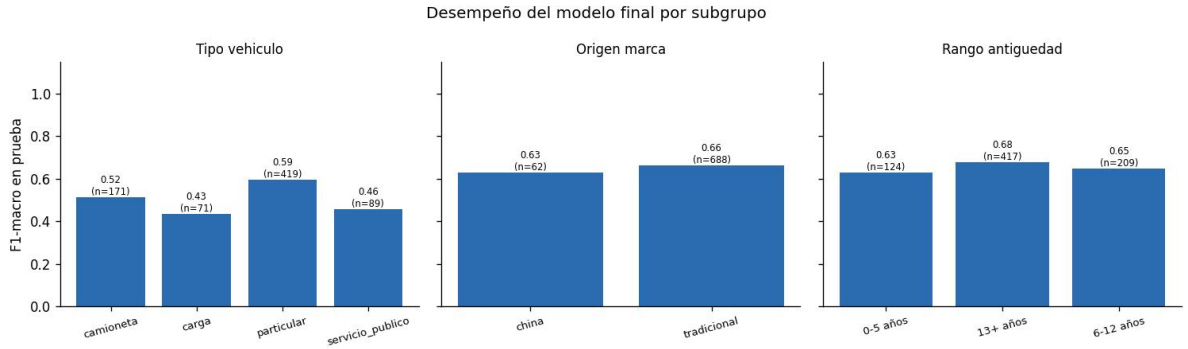


Figura 4. *F1-macro del modelo final por subgrupo en el conjunto de prueba (n = casos del grupo).*

Tabla 6. *Desempeño del modelo final por tipo de vehículo (conjunto de prueba).*

Tipo de vehículo	Casos	Exactitud	F1-macro
Camioneta	171	0,614	0,515
Carga	71	0,648	0,434
Particular	419	0,723	0,595
Servicio público	89	0,607	0,457

Por **tipo de vehículo** hay una brecha clara: la exactitud va de 0,723 en particulares a 0,607 en servicio público, y el F1-macro baja hasta 0,434 en vehículos de carga (brecha máxima de F1-macro: 0,160). Por **origen de la marca** la diferencia es pequeña (brecha de exactitud de 0,000), y por **antigüedad** los vehículos nuevos (0–5 años) tienen un desempeño algo menor (0,629 frente a 0,700), probablemente porque en ellos es más frecuente la clase "sin falla", que es la más difícil de separar.

Hay una precaución al leer el F1-macro por grupo: dentro de cada tipo de vehículo algunas clases casi no aparecen (por ejemplo, casi ningún particular tiene muelles), y un solo error en una clase con dos o tres casos reduce mucho el promedio. Por eso se reporta también la exactitud, y la conclusión se mantiene con ambas métricas.

5.3 Origen del sesgo y prueba de mitigación

Se identificaron dos causas posibles, ambas descritas por Mehrabi et al. (2021): el **sesgo de representación** (los vehículos de carga son solo el 10 % de los datos) y el **sesgo de medición** (sus etiquetas son más ruidosas, porque con frecuencia tienen varias fallas a la vez por la sobrecarga). Para separar una de otra se aplicó la re-ponderación de Kamiran y Calders (2012): se entrenó el mismo modelo dando a cada tipo de vehículo el mismo peso total.

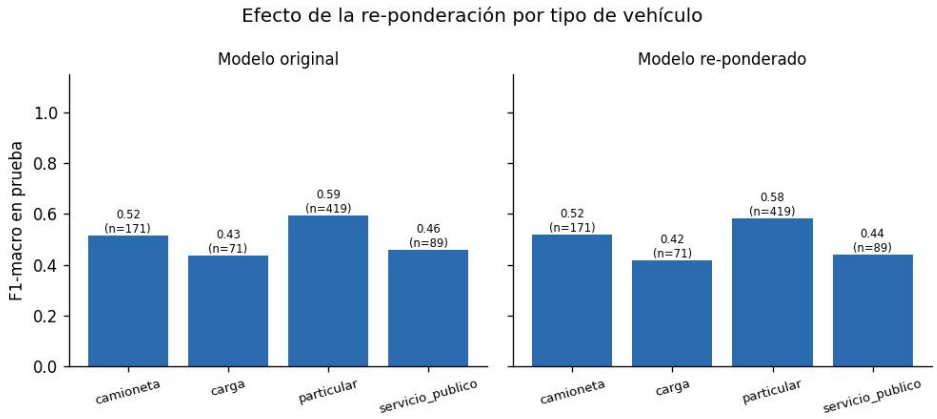


Figura 5. *F1-macro por tipo de vehículo antes y después de la re-ponderación.*

Tabla 7. *Efecto de la re-ponderación por tipo de vehículo (conjunto de prueba).*

Indicador	Modelo original	Re-ponderado
Exactitud global	0,677	0,671
F1-macro global	0,665	0,655
Exactitud en carga	0,648	0,634
Brecha de exactitud entre tipos	0,116	0,104
Brecha de F1-macro entre tipos	0,160	0,167

La re-ponderación no cerró la brecha: la brecha de exactitud entre tipos de vehículo apenas bajó (0,116 → 0,104), la de F1-macro aumentó (0,160 → 0,167) y el desempeño global bajó levemente. Este resultado negativo es informativo: si el problema fuera solo la falta de ejemplos, darles más peso habría ayudado. Que no ayude indica que la causa dominante es la **calidad de las etiquetas** de esos grupos (el ruido del 12 % en carga), y que la solución no está en el

algoritmo sino en los datos: recolectar más casos reales de vehículos de carga y servicio público, con diagnósticos confirmados por un técnico. Mientras tanto, la medida de mitigación adoptada es **operativa**: la API envía siempre a revisión técnica los casos de vehículos de carga y los de baja confianza.

5.4 Implicaciones éticas

- **Seguridad vial:** un diagnóstico errado puede llevar a no reparar una pieza crítica, como una rótula. Por eso el sistema es de **apoyo a la decisión** y nunca reemplaza la inspección: cada respuesta incluye una advertencia, y los casos inciertos se derivan a un técnico (Amershi et al., 2019).
- **Conflicto de interés comercial:** un sistema que recomienda repuestos en una tienda podría sesgarse hacia la venta. Se establece como regla que la clase "sin falla de suspensión" sea una salida válida y visible, y que el modelo no se optimice por ingresos sino por acierto diagnóstico.
- **Transparencia:** el cliente ve las tres hipótesis con su probabilidad, no un veredicto único. Se recomienda publicar una ficha del modelo (Mitchell et al., 2019) con su uso previsto, sus límites y sus resultados por subgrupo.
- **Privacidad y protección de datos:** la Ley 1581 de 2012 (Habeas Data) exige una autorización previa, expresa e informada para tratar datos personales. El modelo no usa nombre, cédula ni teléfono; la placa y los datos de contacto, si se recogen para el seguimiento comercial, deben guardarse aparte del conjunto de entrenamiento, con una finalidad declarada y la posibilidad de que el titular los consulte o pida suprimirlos.
- **Honestidad sobre los datos:** los resultados provienen de datos semisintéticos. Presentarlos como el desempeño real sería engañoso; se reportan como una validación de la metodología.

6. Conclusiones y recomendaciones

6.1 Conclusiones

1. Se construyó un sistema completo de diagnóstico preliminar —datos, modelo, evaluación, análisis de equidad y API— que responde a un problema real de Automuelles Virtual. El modelo final (LightGBM ajustado con Optuna) alcanzó en prueba una exactitud de 67,7 %, un F1-macro de 0,665 y una **exactitud top-3 de 92,5 %**.
2. El ajuste bayesiano de hiperparámetros mejoró el F1-macro y produjo un modelo más regularizado. La pequeña diferencia con la regresión logística muestra la importancia de comparar siempre contra una línea base simple.
3. Las confusiones del modelo coinciden con las ambigüedades reales del diagnóstico por síntomas (bujes frente a terminales, espiral frente a amortiguador), lo que justifica presentar tres hipótesis en lugar de una.
4. El modelo es menos confiable para vehículos de carga y de servicio público. La re-ponderación no corrigió la brecha, lo que apunta a la calidad de los datos de esos grupos como la causa principal.

6.2 Recomendaciones

- **Datos reales:** digitalizar las órdenes de servicio del taller con un formato fijo (vehículo, síntomas reportados y falla confirmada por el técnico) y reentrenar el modelo cuando haya al menos 1.000 casos reales.
- **Ciclo de retroalimentación:** registrar el diagnóstico final del técnico para cada consulta hecha en la plataforma; esto genera etiquetas confiables de forma continua.
- **Prioridad a los grupos con menor desempeño:** recolectar de forma dirigida casos de vehículos de carga y de servicio público.
- **Calibración y umbrales:** calibrar las probabilidades y ajustar el umbral de derivación al técnico con datos reales, midiendo el efecto sobre las devoluciones.
- **Explicabilidad:** agregar explicaciones por caso (por ejemplo, con SHAP) para que el asesor vea qué síntomas pesaron en el diagnóstico.
- **Indicadores de negocio:** medir la tasa de devoluciones, el tiempo de atención y la conversión antes y después de desplegar el sistema.

Referencias

Akiba, T., Sano, S., Yanase, T., Ohta, T. y Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. En *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (pp. 2623–2631). ACM.

Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., ... Horvitz, E. (2019). Guidelines for human-AI interaction. En *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (pp. 1–13). ACM.

Barocas, S., Hardt, M. y Narayanan, A. (2023). *Fairness and machine learning: Limitations and opportunities*. MIT Press.

Bergstra, J. y Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research, 13*, 281–305.

Bird, S., Dudík, M., Edgar, R., Horn, B., Lutz, R., Milan, V., ... Walker, K. (2020). *Fairlearn: A toolkit for assessing and improving fairness in AI* (MSR-TR-2020-32). Microsoft.

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32.

Chawla, N. V., Bowyer, K. W., Hall, L. O. y Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research, 16*, 321–357.

Chen, T. y Guestrin, C. (2016). XGBoost: A scalable tree boosting system. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM.

Congreso de la República de Colombia. (2012). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587.

Grinsztajn, L., Oyallon, E. y Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? En *Advances in Neural Information Processing Systems 35* (Datasets and Benchmarks Track).

Kamiran, F. y Calders, T. (2012). Data preprocessing techniques for classification without discrimination. *Knowledge and Information Systems, 33*(1), 1–33.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. y Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. En *Advances in Neural Information Processing Systems 30* (pp. 3146–3154).

Koren, Y., Bell, R. y Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer, 42*(8), 30–37.

Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K. y Galstyan, A. (2021). A survey on bias and fairness in machine learning. *ACM Computing Surveys, 54*(6), 1–35.

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., ... Gebru, T. (2019). Model cards for model reporting. En *Proceedings of the Conference on Fairness, Accountability, and Transparency* (pp. 220–229). ACM.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Salton, G. y Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management, 24*(5), 513–523.

Shwartz-Ziv, R. y Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion, 81*, 84–90.

Theissler, A., Pérez-Velázquez, J., Kettelgerdes, M. y Elger, G. (2021). Predictive maintenance enabled by machine learning: Use cases and challenges in the automotive industry. *Reliability Engineering & System Safety, 215*, 107864.